



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Muhammad Ilham Fajri
NPM	2428240082
Kelas	SI5B
Topik Tugas 1	Asuransi Resource: Polis
Entitas utama	Polis Asuransi
Tautan repositori	https://github.com/IlhamFajrii/tugas1-restful-2428240082.git
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	Kamis, 1 Oktober 2026, 23:54

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Asuransi Resource: Polis
Entitas utama (tunggal)	Polis Asuransi
Alamat dasar	/api/insurance-policies
Field data	id, nomorPolis, namaPemegang, jenis, premiBulanan, tanggalMulai
Field wajib saat POST	nomorPolis, namaPemegang, jenis, premiBulanan, dan tanggalMulai
Query string untuk filter	Tidak digunakan pada implementasi ini.
Data awal	Data awal terdiri dari dua polis, yaitu POL-001 atas nama Muhammad Ilham Fajri dengan jenis Kesehatan dan POL-002 atas nama Andi Saputra dengan jenis Jiwa.
Modul lain (jika ada)	Tidak ada modul atau relasi antartabel tambahan. Aplikasi menggunakan Model, Controller, Route, dan Middleware.

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/api/insurance-policies	Tidak	200	Mengambil seluruh data polis
2	GET	/api/insurance-policies/:id	Tidak	200	Mengambil satu polis berdasarkan ID
3	POST	/api/insurance-policies	Ya	201	Menambahkan polis baru
4	PUT	/api/insurance-policies/:id	Ya	200	Mengubah data polis berdasarkan ID
5	DELETE	/api/insurance-policies/:id	Ya	204	Menghapus data polis berdasarkan ID

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Jelaskan fungsi *express*, *cors*, dan *dotenv*, serta mengapa versi *Node.js* perlu memenuhi syarat *Express 5*.

Express digunakan sebagai framework Node.js untuk membuat server dan RESTful API pada aplikasi Polis Asuransi. *cors* digunakan agar server dapat menerima request dari origin yang berbeda, sedangkan *dotenv* digunakan untuk membaca konfigurasi dari file `.env`, seperti `PORT` dan `API_KEY`, tanpa menuliskan nilai konfigurasi tersebut langsung di dalam kode.

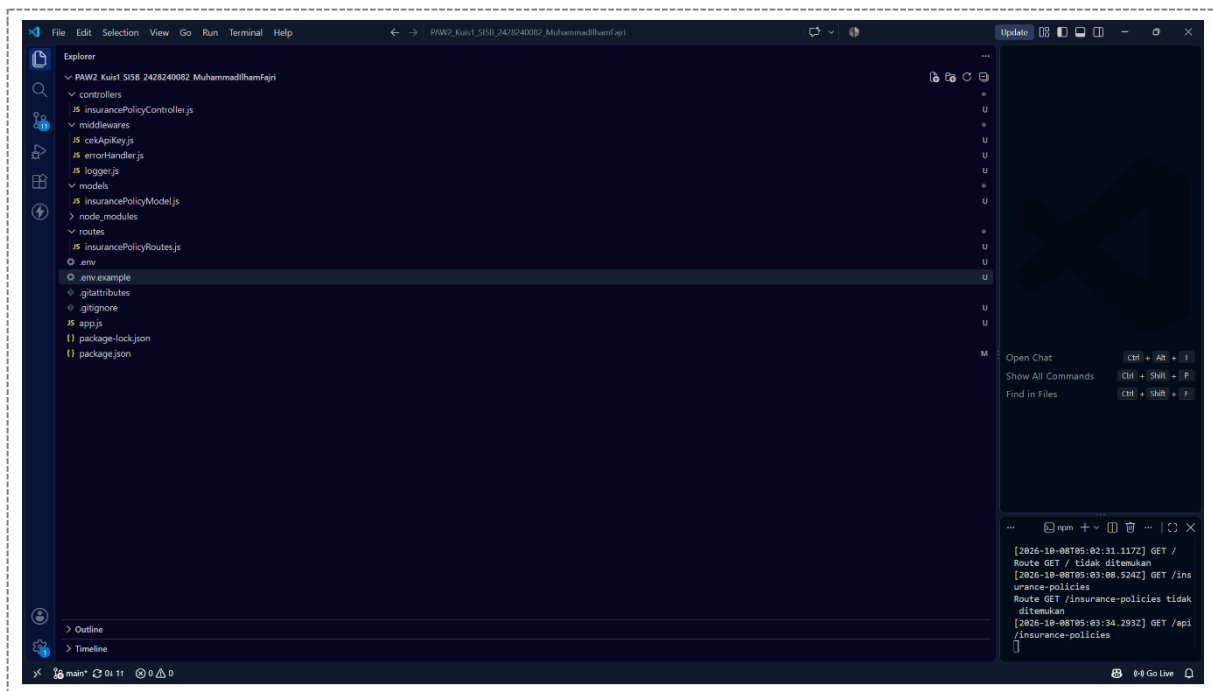
Versi Node.js perlu diperhatikan karena project menggunakan Express 5. Dengan menggunakan versi Node.js yang memenuhi persyaratan Express 5, fitur dan dependensi Express dapat berjalan dengan baik. Pada proses instalasi saya juga membuat project menggunakan `npm init -y` dan memasang dependensi yang dibutuhkan.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Jelaskan tugas setiap folder dalam satu kalimat.

Folder models bertugas mengelola data polis dan menyediakan fungsi untuk mengambil, menambah, mengubah, serta menghapus data. Folder controllers menangani logika request dan response dari endpoint Polis Asuransi. Folder routes menentukan alamat endpoint dan menghubungkan request dengan Controller. Folder middlewares berisi fungsi perantara seperti pencatatan request, pemeriksaan API key, dan penanganan error.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

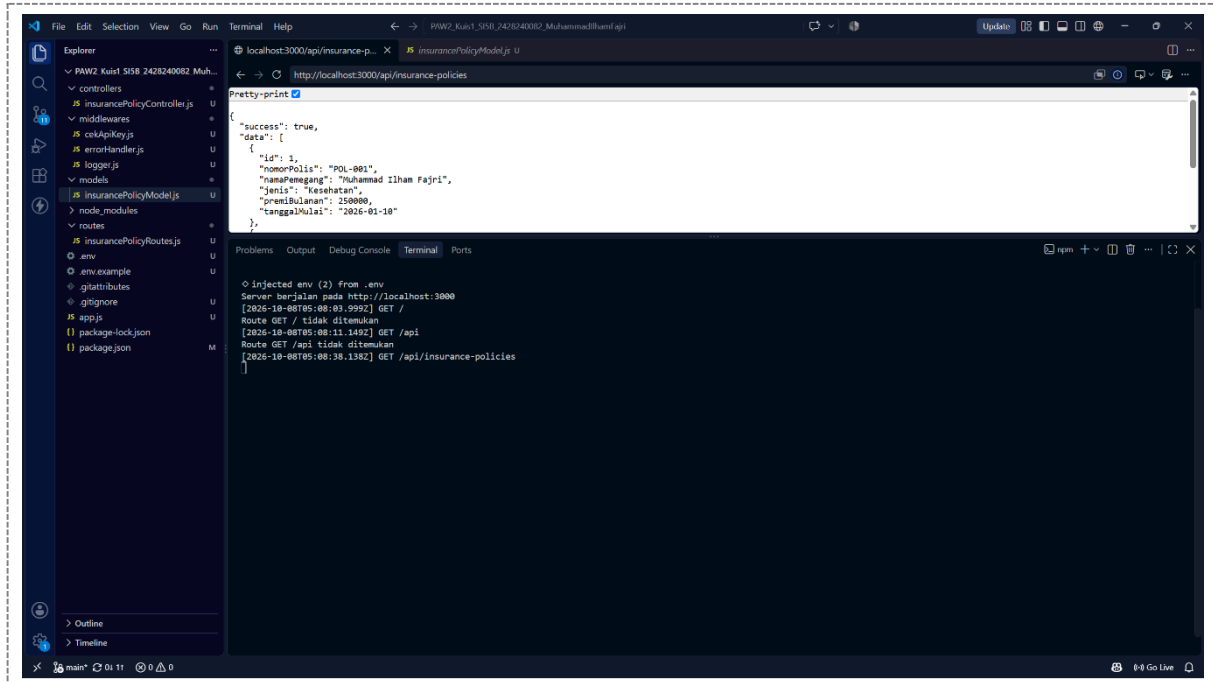
Berkas	Penjelasan Anda
models/insurancePolicyModel.js	File ini bertanggung jawab mengelola data polis asuransi pada variabel insurancePolicies. Fungsi getAll, getById, create, update, dan remove digunakan Controller untuk mengambil, mencari, menambah, mengubah, dan menghapus data polis.

Berkas	Penjelasan Anda
controllers/insurancePolicyController.js	File ini berisi fungsi getAll, getById, create, update, dan remove yang menangani request dari client. Controller juga melakukan validasi seperti memastikan ID berupa angka, data POST lengkap, serta premiBulanan memiliki nilai yang valid sebelum memanggil Model.
routes/insurancePolicyRoutes.js	File ini menentukan endpoint untuk resource Polis Asuransi. Route GET diarahkan ke Controller untuk mengambil data, sedangkan POST, PUT, dan DELETE menggunakan middleware cekApiKey sebelum request diteruskan ke Controller.
middlewares/logger.js	Middleware logger mencatat waktu, metode HTTP, dan alamat request menggunakan req.method dan req.originalUrl. Setelah mencatat request, middleware menjalankan next() agar request dapat diteruskan ke proses berikutnya.
middlewares/cekApiKey.js	Middleware cekApiKey memeriksa header x-api-key pada request. Jika API key tidak ada maka middleware mengembalikan error 401, sedangkan API key yang tidak sesuai dengan nilai API_KEY pada .env menghasilkan error 403.
middlewares/errorHandler.js	File ini menjadi penanganan error terpusat untuk aplikasi. Middleware mengambil err.status atau menggunakan status 500 sebagai default, kemudian mengirimkan response JSON dengan properti success: false dan pesan error.
app.js	File app.js merupakan entry point aplikasi Express. File ini memuat konfigurasi .env, memasang middleware CORS, logger, dan express.json(), menghubungkan route /api/insurance-policies, menangani route yang tidak ditemukan, memasang errorHandler, dan menjalankan server menggunakan port dari environment variable.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Penjelasan:

Jelaskan dari mana nomor port berasal dan apa yang terjadi bila port sudah dipakai aplikasi lain.

Nomor port server berasal dari `process.env.PORT` yang dibaca melalui file `.env`. Pada project ini nilai `PORT` adalah 3000, sehingga server berjalan pada `http://localhost:3000`. Jika environment variable `PORT` tidak tersedia, kode menggunakan nilai cadangan 3000 melalui `process.env.PORT || 3000`.

Jika port yang digunakan sudah dipakai aplikasi lain, server tidak dapat melakukan binding pada port tersebut dan Node.js akan menghasilkan error bahwa port sedang digunakan. Dalam kondisi tersebut, aplikasi lain harus dihentikan atau nomor port pada konfigurasi harus diganti.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

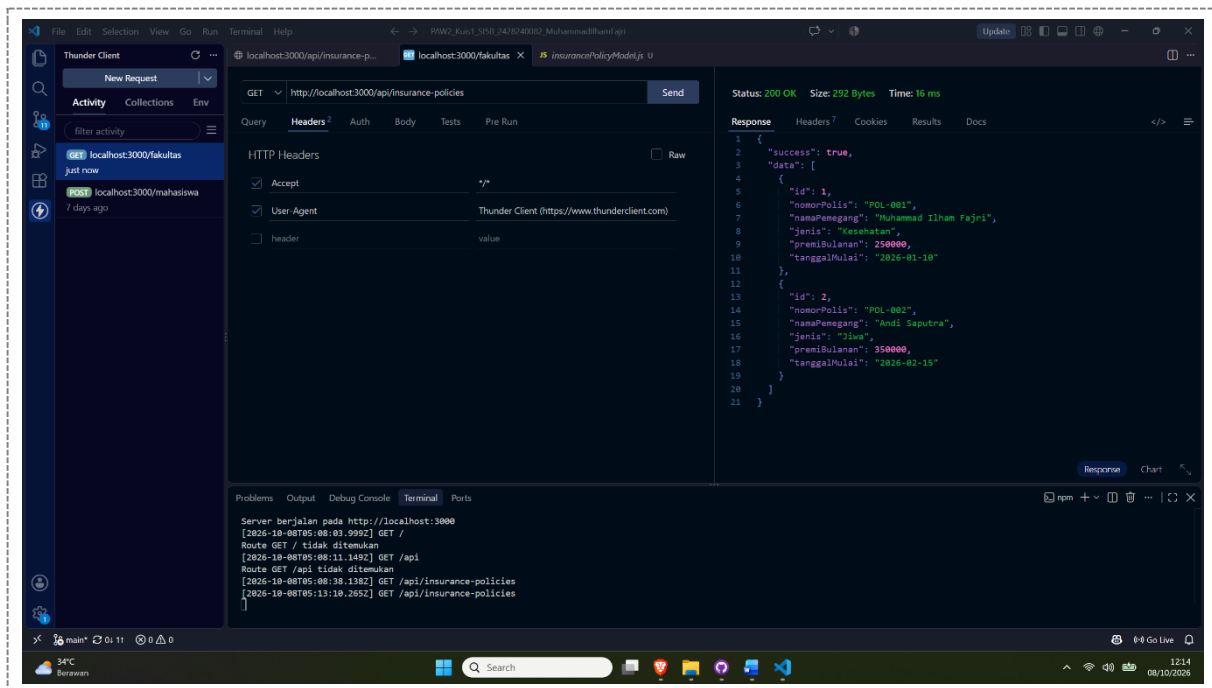
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[sumber-daya])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET http://localhost:3000/api/insurance-policies
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK - success: true, data berisi seluruh data polis
Lulus?	☒ Ya ☐ Tidak



Gambar 4. Hasil GET semua data

Penjelasan:

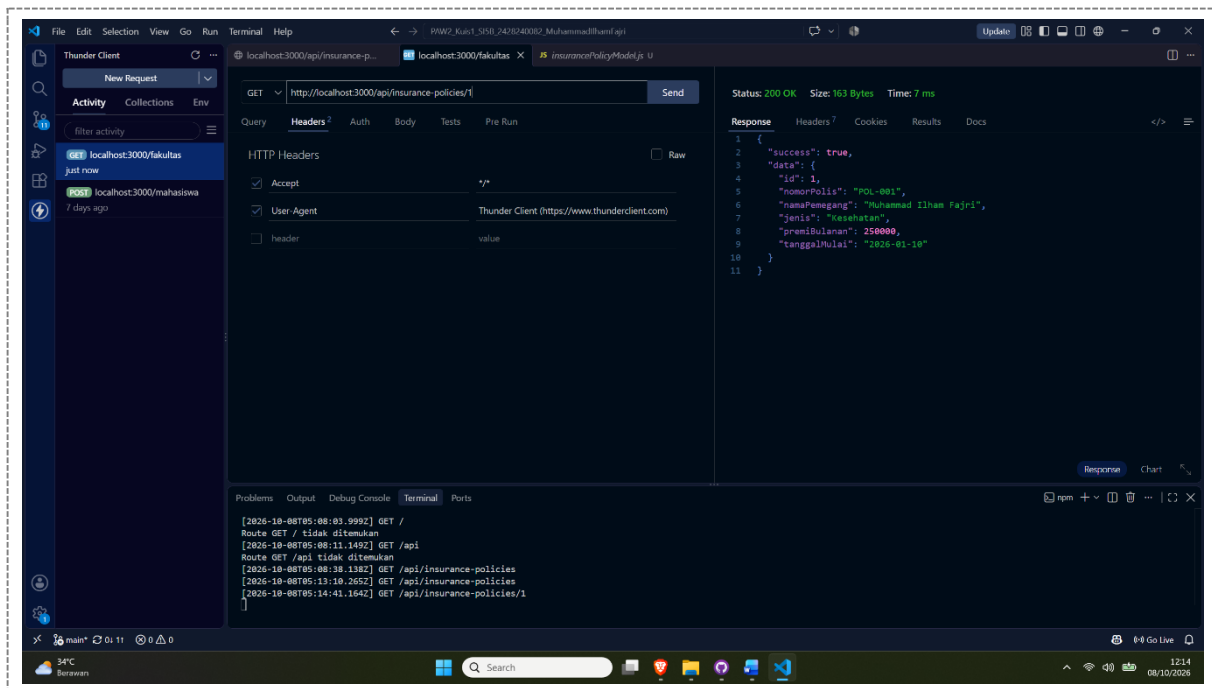
Jelaskan kode status yang muncul dan kemampuan endpoint ini. Sebutkan urutan pemanggilan: route mana, fungsi Controller mana, dan fungsi Model mana yang bekerja.

Request GET `/api/insurance-policies` pertama kali masuk ke route `router.get("/", controller.getAll)`. Route kemudian meneruskan request ke fungsi `getAll` pada `insurancePolicyController.js`. Controller memanggil `insurancePolicyModel.getAll()` untuk mengambil seluruh data pada variabel `insurancePolicies`, kemudian hasilnya dikirim sebagai response JSON dengan status 200

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET <code>/[sumber-hari]/[id yang ada]</code>
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK dan response berisi satu objek polis dengan ID 1.
Status hasil uji	(200 OK - success: true, data berisi polis dengan id 1)
Lulus?	☒ Ya ☐ Tidak



Gambar 5. Hasil GET berdasarkan ID

Penjelasan:

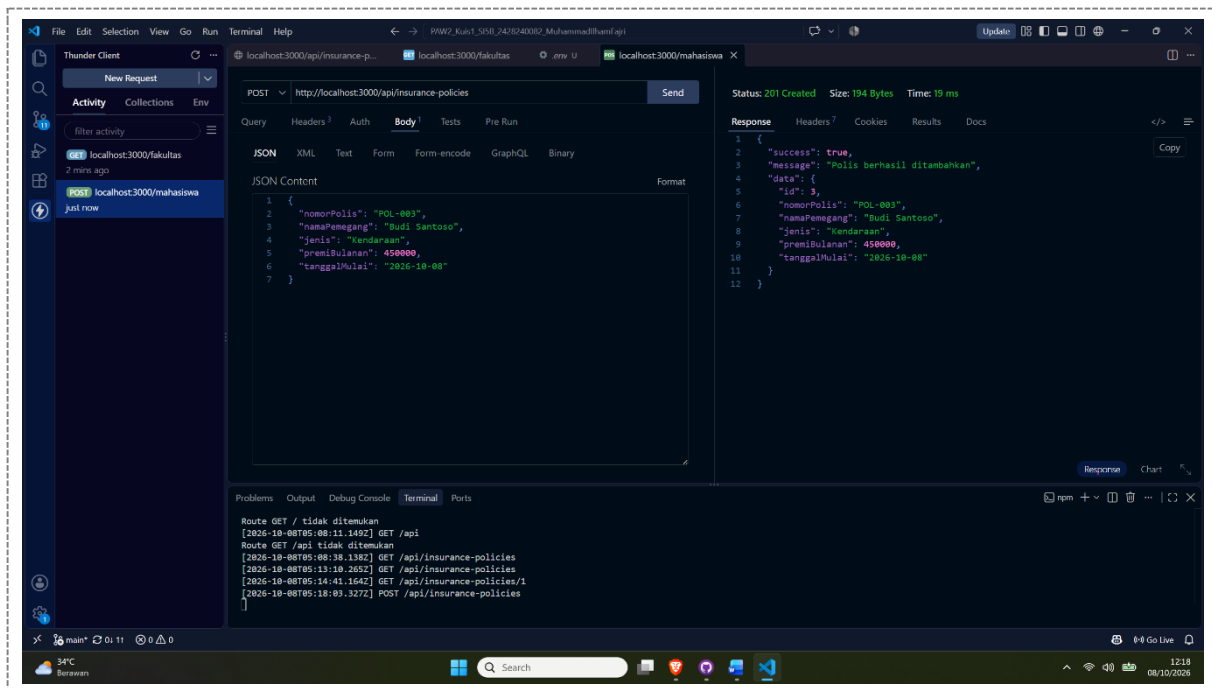
Jelaskan bagaimana ID dari alamat diambil (`req.params`) dan mengapa perlu diubah menjadi angka.

ID dari alamat request diambil menggunakan `req.params.id`. Nilai tersebut pada awalnya berupa string karena berasal dari URL, sehingga pada Controller saya mengubahnya menjadi angka menggunakan `Number(req.params.id)`. Hal ini diperlukan karena ID pada Model disimpan sebagai tipe number, sehingga pencarian menggunakan `insurancePolicyModel.getById(id)` dapat menghasilkan data yang sesuai.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (<i>request</i>)	POST http://localhost:3000/api/insurance-policies
Header	x-api-key: kuis1_ilham_2026
Body	<pre>{ "nomorPolis": "POL-003", "namaPemegang": "Budi Santoso", "jenis": "Kendaraan", "premiBulanan": 450000, "tanggalMulai": "2026-10-08" }</pre>
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created - Polis berhasil ditambahkan
Lulus?	☒ Ya ☐ Tidak



Gambar 6. Hasil POST data baru

Penjelasan:

Jelaskan peran header `x-api-key` (middleware apa yang memeriksanya), peran `express.json()`, dan alasan status yang dipakai adalah 201, bukan 200.

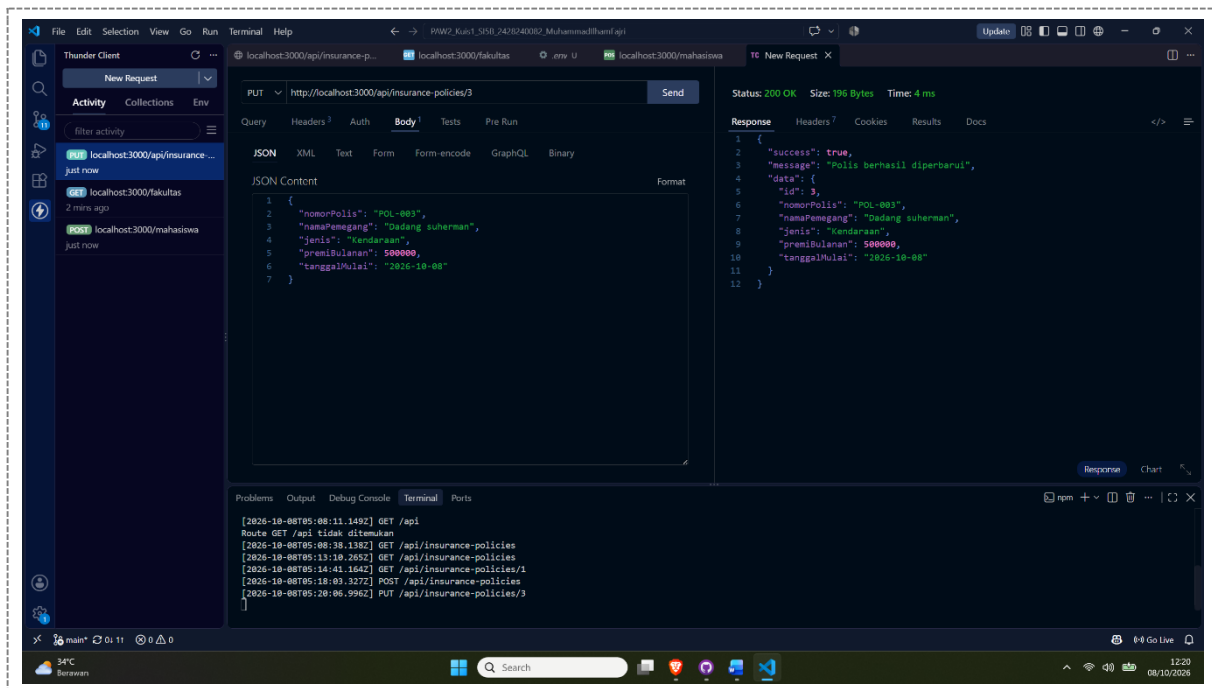
Header `x-api-key` diperiksa oleh middleware `cekApiKey`. Middleware membandingkan nilai API key dari request dengan `process.env.API_KEY`; jika sesuai maka request diteruskan ke Controller. Middleware `express.json()` digunakan untuk membaca body request yang memiliki format JSON sehingga data dapat diakses melalui `req.body`. Setelah data divalidasi, Controller memanggil `insurancePolicyModel.create()` untuk membuat data polis baru.

Status 201 Created digunakan karena request berhasil membuat resource baru, yaitu data Polis Asuransi. Status ini lebih tepat daripada 200 OK karena 200 biasanya digunakan untuk request yang berhasil tetapi tidak secara khusus menunjukkan resource baru telah dibuat.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (<i>request</i>)	PUT http://localhost:3000/api/insurance-policies/3
Header	x-api-key: kuis1_ilham_2026
Body	<pre>{ "nomorPolis": "POL-003", "namaPemegang": "Dadang suherman", "jenis": "Kendaraan", "premiBulanan": 500000, "tanggalMulai": "2026-10-08" }</pre>
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	☒ Ya ☐ Tidak



Gambar 7. Hasil PUT mengubah data

Penjelasan:

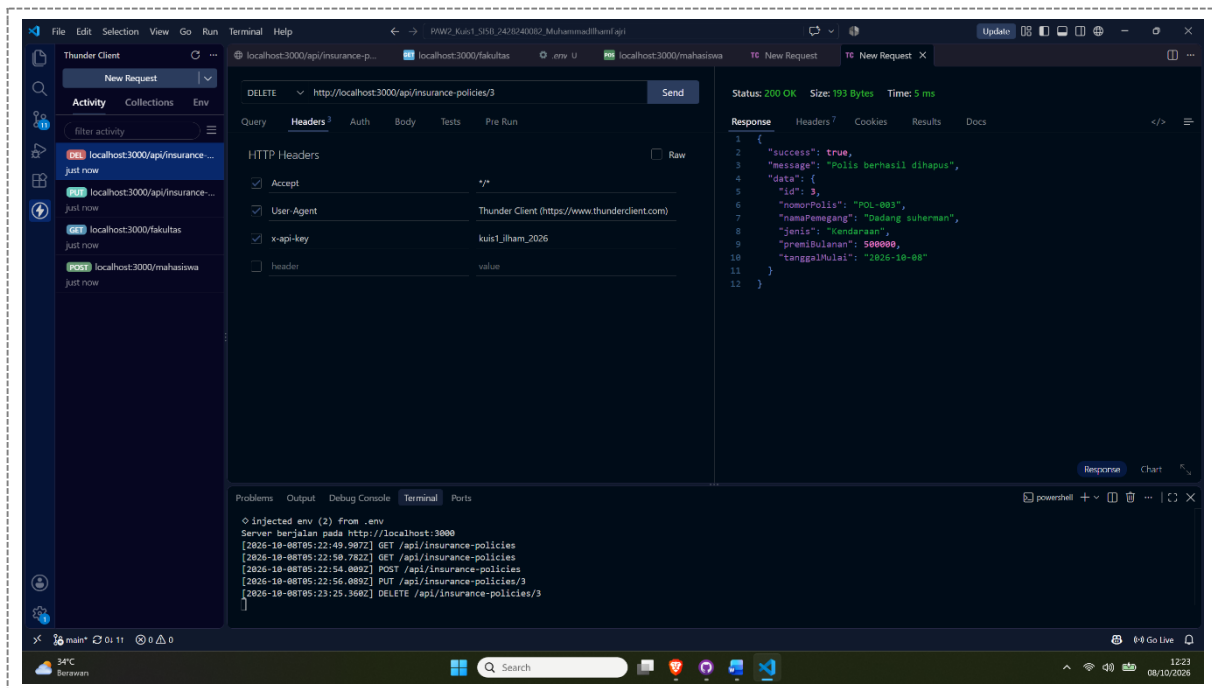
Jelaskan field apa yang berubah dan field apa yang tetap, serta layer mana yang melakukan perubahan.

Pada pengujian ini field premiBulanan diubah dari 450000 menjadi 500000. Field nomorPolis, namaPemegang, jenis, dan tanggalMulai tetap menggunakan nilai yang dikirim pada request. Perubahan dilakukan melalui alur Route → Controller → Model. Controller memeriksa ID dan data request terlebih dahulu, kemudian memanggil fungsi update() pada Model. Model mencari index data berdasarkan ID dan menggabungkan data lama dengan data baru menggunakan spread operator.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE <code>http://localhost:3000/api/insurance-policies/3</code>
Header	x-api-key: kuis1_ilham_2026
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	200 OK data terhapus
Lulus?	[] Ya [x] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Jelaskan arti status 204 dan mengapa respons tidak berisi apa pun.

Status 204 No Content menunjukkan bahwa request berhasil diproses tetapi server tidak mengirimkan isi response. Status ini sesuai untuk operasi DELETE ketika data berhasil dihapus dan tidak diperlukan informasi tambahan pada response body.

Pada proses ini Route memeriksa API key melalui middleware `cekApiKey`, kemudian Controller memanggil `insurancePolicyModel.remove()` untuk menghapus data berdasarkan ID. Setelah penghapusan berhasil, server mengembalikan status 204 tanpa body.

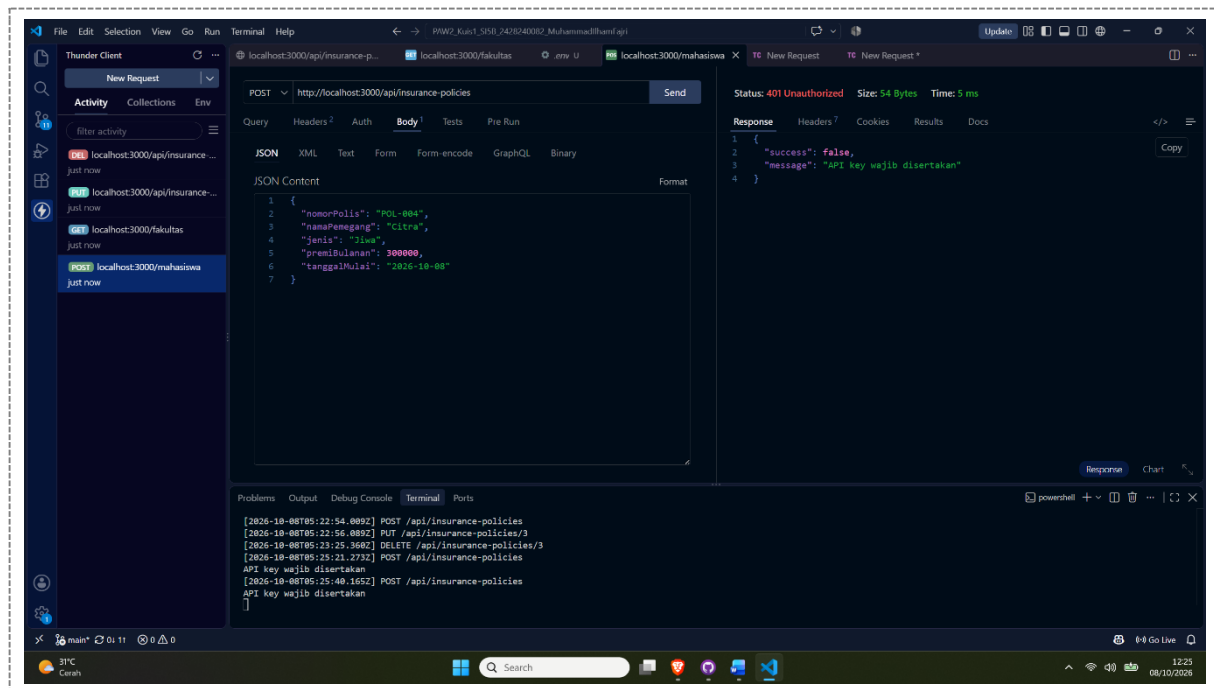
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada **screenshot**: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (<i>request</i>)	POST http://localhost:3000/api/insurance-policies
Header	Content-Type: application/json
Body	<pre>{ "nomorPolis": "POL-004", "namaPemegang": "Citra", "jenis": "Jiwa", "premiBulanan": 300000, "tanggalMulai": "2026-10-08" }</pre>
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized - API key wajib disertakan
Lulus?	☒ Ya ☐ Tidak



Gambar 9. POST tanpa API key ditolak

Penjelasan:

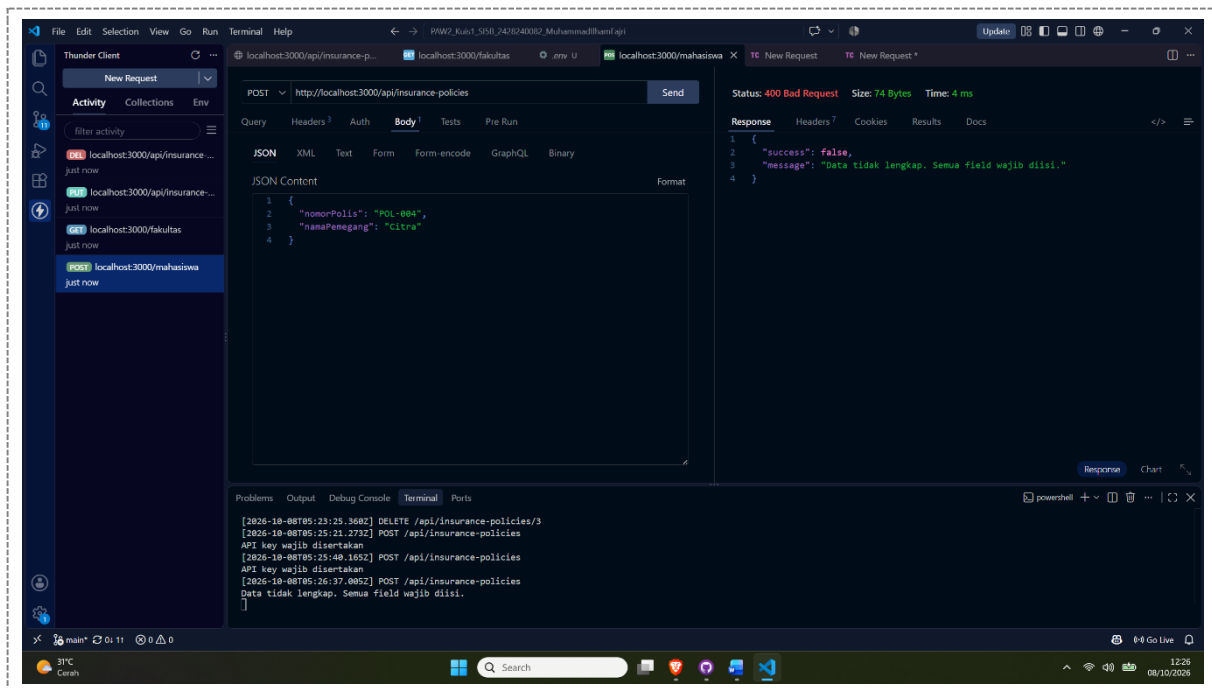
Jelaskan middleware mana yang menolak permintaan dan mengapa Controller tidak sempat dijalankan.

Request ditolak oleh middleware `cekApiKey` sebelum mencapai Controller. Middleware memeriksa `req.headers["x-api-key"]` dan menemukan bahwa header tersebut tidak tersedia. Middleware kemudian membuat error dengan status 401 dan meneruskannya menggunakan `next(error)`. Karena Controller belum dijalankan, tidak ada data baru yang ditambahkan ke Model.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST <code>http://localhost:3000/api/insurance-policies</code>
Header	<code>x-api-key: kuis1_ilham_2026</code>
Body	<pre>{ "nomorPolis": "POL-004", "namaPemegang": "Citra" }</pre>
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[☒] Ya [☐] Tidak



Gambar 10. POST dengan body tidak lengkap

Penjelasan:

Jelaskan di layer mana validasi ini ditulis dan mengapa kode statusnya 400.

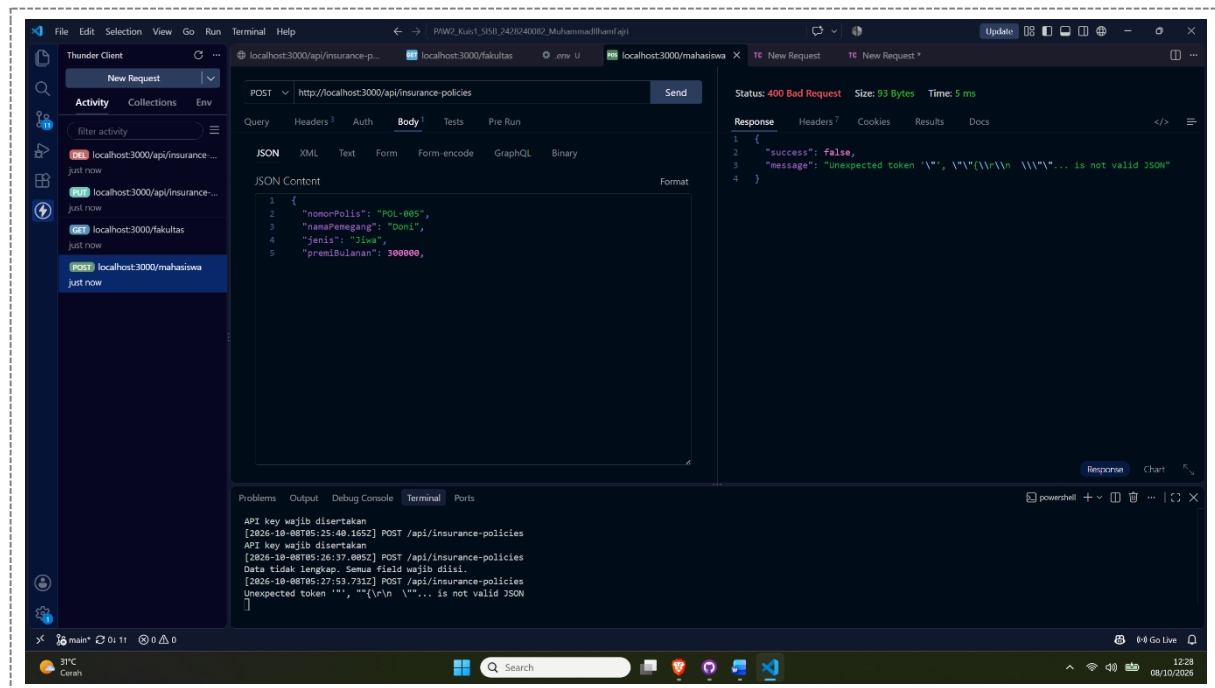
Validasi data ditulis pada layer Controller, tepatnya pada fungsi create. Controller mengambil field nomorPolis, namaPemegang, jenis, premiBulanan, dan tanggalMulai dari req.body, kemudian memeriksa apakah seluruh field wajib sudah tersedia.

Jika ada field yang tidak tersedia, Controller membuat error dengan status 400. Status 400 Bad Request digunakan karena kesalahan berasal dari data request yang dikirim oleh client, bukan dari kegagalan server.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST http://localhost:3000/api/insurance-policies
Header	x-api-key: kuis1_ilham_2026
Body	{ "nomorPolis": "POL-005", "namaPemegang": "Doni", "jenis": "Jiwa", "premiBulanan": 300000,
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request - Format JSON tidak valid
Lulus?	☒ Ya ☐ Tidak



Gambar 11. POST dengan JSON rusak

Penjelasan:

Jelaskan siapa yang mendeteksi JSON rusak ini dan bagaimana error handler mengubahnya menjadi respons yang rapi.

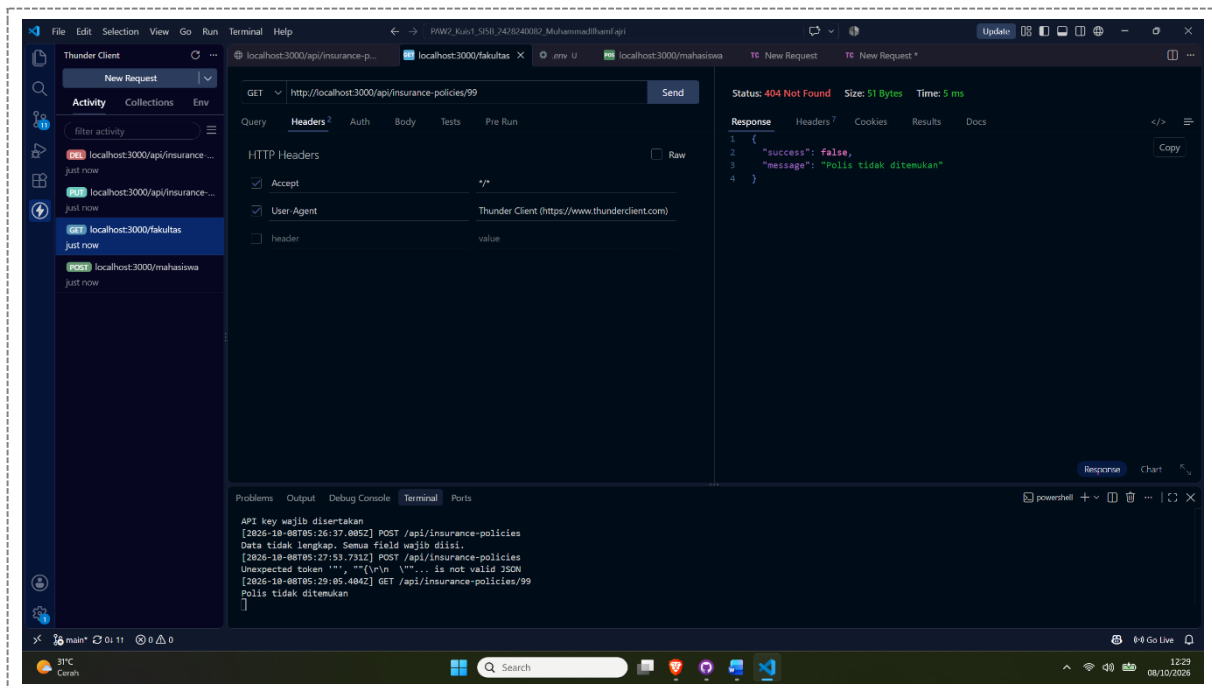
JSON yang rusak dideteksi oleh middleware `express.json()` ketika Express mencoba membaca body request sebagai JSON. Karena terdapat kesalahan sintaks seperti koma berlebih, proses parsing JSON menghasilkan error.

Error tersebut kemudian diteruskan ke error handler terpusat. Middleware errorHandler mengubah error menjadi response JSON sehingga client tidak menerima halaman HTML atau stack trace. Dengan demikian, kesalahan format JSON tetap menghasilkan response API yang konsisten.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (<i>request</i>)	GET http://localhost:3000/api/insurance-policies/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found - Polis tidak ditemukan
Lulus?	☒ Ya ☐ Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Jelaskan alur kode sampai pesan 404 dibuat dan siapa yang mengirimkannya.

Request masuk ke route `router.get('/:id', controller.getById)`. Controller mengambil ID dari `req.params.id`, mengubahnya menjadi angka, kemudian memanggil `insurancePolicyModel.getById(id)`.

Karena ID 99 tidak terdapat pada data Polis Asuransi, Model mengembalikan `undefined`. Controller kemudian membuat error dengan pesan `Polis tidak ditemukan` dan status 404, lalu meneruskannya menggunakan `next(error)`. Error tersebut akhirnya dikirim dalam bentuk JSON oleh `errorHandler`.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200 OK	Ya
T02	Menampilkan satu data berdasarkan ID	200	200 OK	Ya
T03	Menambah data baru	201	201 Created	Ya
T04	Mengubah data	200	200 OK	ya
T05	Menghapus data	204	200 OK data terhapus	tidak
N01	Menambah data tanpa API key	401	401 Unauthorized	Ya
N02	Menambah data dengan body tidak lengkap	400	400 Bad Request	Ya
N03	Mengirim JSON yang rusak	400	400 Bad Request	Ya
N04	Mengambil ID yang tidak ada	404	404 Not Found	Ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/IlhamFajrii/PAW2_Kuis1_SI5B_2428240082_MuhammadIlhamFajri
Pesan commit terakhir	Laporanpdfkuispaw2
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Tulis jawaban Anda di sini.

Route bertugas menentukan alamat endpoint dan menghubungkan request dengan Controller, contohnya `router.get("/", controller.getAll)`. Controller menangani logika request dan response, contohnya fungsi `getAll` yang memanggil `insurancePolicyModel.getAll()`. Model bertugas mengelola data Polis Asuransi, contohnya fungsi `getAll` yang mengembalikan variabel `insurancePolicies`.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Tulis jawaban Anda di sini.

Error handler terpusat membuat penanganan kesalahan menjadi lebih konsisten karena berbagai error dapat diteruskan menggunakan `next(error)` dan diproses oleh satu middleware `errorHandler`. Buktinya terlihat pada pengujian N04 ketika ID 99 tidak ditemukan dan error dari Controller diteruskan ke `errorHandler`, sehingga client menerima response JSON 404 Not Found tanpa perlu menulis response error langsung pada setiap route.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	☒
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	☒
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	☒
4	Identitas saya (nama atau NIM) muncul pada data uji POST	☒
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	☒
6	Tabel rekapitulasi sesuai dengan tangkapan layar	☒
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	☒
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	☒

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100